

```
import uasyncio as asyncio
from machine import Pin, I2C, PWM
from pico_i2c_lcd import I2cLcd
from time import sleep
import random
from picozero import LED, Speaker, Button

#define LEDs
led1 = LED(15)
led2 = LED(14)
led3 = LED(13)
led4 = LED(11)
led5 = LED(16)
led6 = LED(17)
led7 = LED(18)
led8 = LED(19)

#assign a specific color to each LED
red = [led6, led3]
blue = [led8, led5, led2]
green = [led1, led4, led7]

rand_led = [led1, led2, led3, led4, led5, led6, led7,
led8]#random LED

i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=100000)#define i2d
pins
i2c2 = I2C(1, scl=Pin(3), sda=Pin(2), freq=100000)
```

```
lcd1 = I2cLcd(i2c, 0x27, 4, 20)#define LCDs
```

```
lcd2 = I2cLcd(i2c2, 0x27, 4, 20)
```

```
ain1 = Pin(20, Pin.OUT)#Ain and Bin pins of the motor
```

```
ain2 = Pin(5, Pin.OUT)
```

```
bin1 = Pin(8, Pin.OUT)
```

```
bin2 = Pin(7, Pin.OUT)
```

```
total_timeA = 0
```

```
total_timeB = 0
```

```
pwma = PWM(Pin(4)) #Assign PWM motor pin
```

```
pwmb = PWM (Pin(9))
```

```
pwma.freq(500)
```

```
pwmb.freq(500)
```

```
stby = Pin(6, Pin.OUT)#motor's standby pin and value
```

```
stby.value(1)
```

```
winner = None
```

```
player1_points = 0 #set player points = 0
```

```
player2_points = 0
```

```
game_running = False
```

```
def stop(): #function to stop the motors
```

```
    pwma.duty_u16(0)
```

```
    pwmb.duty_u16(0)
```

```
def reset_motorA(A): #function to reset motor A
```

```
    ain1.value(1)
```

```
    ain2.value(0)
```

```
    pwma.duty_u16(30000)
```

```
    sleep(A)
```

```
    stop()
```

```
def reset_motorB(B): #function to reset motor B
```

```
    bin1.value(1)
```

```
    bin2.value(0)
```

```
    pwmb.duty_u16(30000)
```

```
    sleep(B)
```

```
    stop()
```

```
#A class defining which number or a letter corresponds on a  
keypad
```

```
#Also a class that tells the code how to interpret the clicks on  
the keypad
```

```
class I2CKeypad:
```

```
    def __init__(self, i2c, addr):
```

```
        self.i2c    = i2c
```

```
        self.addr   = addr
```

```
        self.keys   = [
```

```

        ['D', '/', '0', '*'],
        ['C', '9', '8', '7'],
        ['B', '6', '5', '4'],
        ['A', '3', '2', '1']
    ]

    self._last_key = None

def write(self, value):
    self.i2c.writeto(self.addr, bytes([value]))

def read(self):
    return self.i2c.readfrom(self.addr, 1)[0]

def scan_raw(self):
    for row in range(4):
        self.write(0xFF & ~(1 << row))
        cols = self.read() >> 4
        for col in range(4):
            if not (cols & (1 << col)):
                return self.keys[row][col]
    return None

async def get_key(self):

    while True:
        key = self.scan_raw()
        if key is not None and key != self._last_key:

```

```

        self._last_key = key
        # Wait for release without blocking
        while self.scan_raw() is not None:
            await asyncio.sleep_ms(10)
        self._last_key = None
        await asyncio.sleep_ms(50)    # settling delay
        return key
    self._last_key = key
    await asyncio.sleep_ms(10)        # yield to other
tasks

```

```

keypad1 = I2CKeypad(i2c, 0x20)#assign keypads
keypad2 = I2CKeypad(i2c2, 0x20)

```

```

def fraction_gen():#A function that generates and prints
fractions

```

```

A = random.randrange(1, 10)
B = random.randrange(2, 10)
C = random.randrange(1, 10)
D = random.randrange(2, 10)

while A >= B or C >= D:
    A = random.randrange(1, 10)
    B = random.randrange(2, 10)
    C = random.randrange(1, 10)

```

```

        D = random.randrange(2, 10)

    print(f"{A}/{B} + {C}/{D}")

    print((A*D + B*C)/(B * D))

    return [f"{A}/{B} + {C}/{D}", (A*D + B*C)/(B * D)]

def clear_row(lcd, row, cols = 20): #a function that clears a
specific row

    lcd.move_to(0, row)

    lcd.putstr(" " * cols)

    lcd.move_to(0, row)

# a function that assigns what player 1 inputs on the keypad
mean

async def player1_input():

    global winner

    answer = ""

    lcd1.move_to(13, 2)

    lcd1.putstr("")

    while True:

        if winner is not None:

            return None

        lcd1.move_to(13, 2)

        key = await keypad1.get_key()

        if key == "D":

            break

        elif key == "C":

```

```

        answer = answer[:-1]

        clear_row(lcd1, 2)

        lcd1.putstr(f"Type answer: ")
    else:
        answer += key

        lcd1.move_to(13, 2)

        lcd1.putstr(answer)

    lcd1.move_to(0, 2)

    clear_row(lcd1, 2)

    lcd1.putstr(f"Got: {answer}")

    winner = 1

    return answer

# a function that assigns what player 2 inputs on the keypad
mean

async def player2_input():
    answer = ""

    lcd2.move_to(13, 2)

    lcd2.putstr("")

    global winner

    while True:
        if winner is not None:
            return None

        lcd2.move_to(13, 2)

        key = await keypad2.get_key()

        if key == "D":

```

```
        break
    elif key == "C":
        answer = answer[:-1]

        clear_row(lcd2, 2)

        lcd2.putstr(f"Type answer: ")
    else:
        answer += key

        lcd2.move_to(13, 2)

        lcd2.putstr(answer)

    lcd2.move_to(0, 2)

    clear_row(lcd2, 2)

    lcd2.putstr(f"Got: {answer}")

    winner = 2

    return answer
```

```
async def player1_confirmation(): #waits for both of the players
to press A to start the game
```

```
    while True:

        key = await keypad1.get_key()

        if key == "A":

            return 1
```

```
async def player2_confirmation():
```

```
    while True:

        key = await keypad2.get_key()

        if key == "A":

            return 2
```



```
def parse_answer(answer : str): #analyzes input from keypads and
evaluates the players' answers
```

```
    answers = answer.split("/")
```

```
    print(answers)
```

```
    print(int(answers[0])/int(answers[1]))
```

```
    return int(answers[0])/int(answers[1])
```

```
#compares the player input to the actual result and checks if it
is correct
```

```
def compare_result(player_result, actual_result):
```

```
    if abs(actual_result - player_result) < 0.0001:
```

```
        return True
```

```
    return False
```

```
def leds_off(): #turns off leds
```

```
    for led in rand_led:
```

```
        led.off()
```

```
def roulette():#"spins" the leds a random amount of times and
checks which led it lands on
```

```
    random_num = random.randint(40, 100)
```

```
    last_led = None
```

```
    for i in range(0, random_num + 1):
```

```
        leds_off()
```

```
        rand_led[i % 8].on()
```

```
        last_led = rand_led[i % 8]
```

```

        sleep(0.07)
    if last_led in red:
        return 3
    elif last_led in blue:
        return 2
    else:
        return 1

toggle_state = False

def toggle(): # power on button
    global game_running
    game_running = not game_running

    lcd1.clear()
    lcd2.clear()

    if game_running:
        lcd1.putstr("Game Starting...")
        lcd2.putstr("Game Starting...")
    else:
        lcd1.putstr("Game Stopped")
        lcd2.putstr("Game Stopped")
        stop() # stop motors

button = Button(22)
button.when_pressed = toggle

```

```

async def main():
    stop()
    global total_timeA
    global total_timeB
    global winner
    global player1_points
    global player2_points
    global game_running

    lcd1.clear()
    lcd2.clear()

    lcd1.putstr("Player 1: Press A") #press A to start
    lcd2.putstr("Player 2: Press A")

    await asyncio.gather( #waits for both players to confirm
start
        player1_confirmation(),
        player2_confirmation()
    )

    while True:
        lcd1.clear()
        lcd2.clear()
        winner = None
        lcd1.move_to(0,1)

```

```

lcd2.move_to(0,1)

fraction_generator = fraction_gen()

lcd1.putstr(f"Solve: {fraction_generator[0]}!") #display
fractions

lcd2.putstr(f"Solve: {fraction_generator[0]}!")

lcd1.move_to(0, 2)
lcd2.move_to(0, 2)

lcd1.putstr("Type answer: ")
lcd2.putstr("Type answer: ")

task1 = asyncio.create_task(player1_input())
task2 = asyncio.create_task(player2_input())


# Wait until someone wins
while winner is None:
    await asyncio.sleep_ms(10)


first_player = winner


task1.cancel()
task2.cancel()


# Safely get results
try:
    answer1 = await task1
except:
    answer1 = None

```

```

try:
    answer2 = await task2
except:
    answer2 = None

actual_answer = fraction_generator[1]

    if winner == 1: #check if the first or the second
player got the answer right first
        first_answer = answer1
        second_task = player2_input
        second_lcd = lcd2
        first_lcd = lcd1
        second_player = 2
elif winner == 2:
        first_answer = answer2
        second_task = player1_input
        second_lcd = lcd1
        first_lcd = lcd2
        second_player = 1

    first_correct = False

#if a player doesn't get it right on the first try, let the
other player try

    if first_answer is not None:
        try:
            ans = parse_answer(first_answer)
            first_correct = compare_result(ans,
actual_answer)
        except:

```

```

        first_correct = False

    if first_correct: # if the first answer was correct,
spin the spinner, add points and move the player

        if winner == 1:

            roulette_spin = roulette()

            player1_points += roulette_spin

            ain1.value(0)

            ain2.value(1)

            pwma.duty_u16(30000)

            sleep(0.055* roulette_spin)

            total_timeA += 0.055 * roulette_spin

            stop()

        else:

            roulette_spin = roulette()

            player2_points += roulette_spin

            bin1.value(0)

            bin2.value(1)

            pwmb.duty_u16(30000)

            sleep(0.055 * roulette_spin)

            total_timeB += 0.055 * roulette_spin

            stop()


    first_lcd.move_to(0, 3)

    first_lcd.putstr("Correct") #display correct

    else:# if the first answer was incorrect, move one to
the second player

        first_lcd.move_to(0, 3)

        first_lcd.putstr("Wrong!")

```

```

# Let second player answer SAME question
winner = None # reset winner so second can play
second_lcd.move_to(0, 2)
clear_row(second_lcd, 2)
second_lcd.putstr("Your turn!")
answer_second = await second_task()
if answer_second is not None:
    try:
        ans2 = parse_answer(answer_second)
        second_correct = compare_result(ans2,
actual_answer)
    except:
        second_correct = False
if second_correct:
    if second_player == 1:
        roulette_spin = roulette()
        player1_points += roulette_spin
        ain1.value(0)
        ain2.value(1)
        pwma.duty_u16(30000)
        sleep(0.055 * roulette_spin)
        total_timeA += 0.055 * roulette_spin
        stop()
    else:
        roulette_spin = roulette()
        player2_points += roulette_spin
        bin1.value(0)

```

```

        bin2.value(1)

        pwmb.duty_u16(30000)

        sleep(0.055 * roulette_spin)

        total_timeB += 0.055 * roulette_spin

        stop()

        second_lcd.move_to(0, 3)

        second_lcd.putstr("Correct!")

    else:

        second_lcd.move_to(0, 3)

        second_lcd.putstr("Wrong!")

    if player1_points >= 12 or player2_points >= 12: # set
up a winning condition

        sleep(1.5)

        clear_row(lcd1, 3)

        clear_row(lcd2, 3)

        lcd1.clear()

        lcd2.clear()

        if player1_points == 12:

            lcd1.putstr("PLAYER 1 WINS!")

            lcd2.putstr("PLAYER 1 WINS!")

        else:

            lcd1.putstr("PLAYER 2 WINS!")

            lcd2.putstr("PLAYER 2 WINS!")

    sleep(3) #reset the game

```



```
reset_motorA(total_timeA)
reset_motorB(total_timeB)
```

```
player1_points = 0
player2_points = 0
```

```
lcd1.clear()
lcd2.clear()
lcd1.putstr("Player 1 Ready")
lcd2.putstr("Player 2 Ready")
continue
```

```
asyncio.run(main())
```